Active Directory RID & Group-Provisioning Capacity Assessment

Implementation Solution Guide

Version 1.0.0 · 2026-07-03

Contents

Executive Summary 4

The decision . . . 4

Why "will we run out of RIDs?" is the wrong first question . . . 4

The risks that actually matter . . . 4

Business value . . . 4

Solution Overview 5

What this assessment is . . . 5

Why teams ask the wrong question first . . . 5

RID fundamentals (the substance you are measuring) . . . 5

What actually drives consumption . . . 6

The decision this assessment supports . . . 6

Requirements & Inputs 6

R1. Active Directory read access (domain-wide) . . . 6

R2. RSAT ActiveDirectory PowerShell module . . . 7

R3. RID Master reachability and health . . . 7

R4. The provisioning rate and groups-per-object count . . . 7

R5. Dedicated OU and naming standard for provisioned groups . . . 8

R6. Ownership metadata on provisioned groups . . . 8

R7. Token-size and RID monitoring tooling . . . 9

Assessment Methodology 9

Step 1 — Measure current RID consumption and RID Master health . . . 9

Step 2 — Project the pool lifetime . . . 9

Step 3 — Evaluate the non-RID risks (the ones that actually matter) . . . 10

Step 4 — Review the authorization architecture . . . 12

Step 5 — Produce findings and conditions of approval . . . 12

Testing & Validation 13

How to run the gates . . . 13

Gate 1 — RID Master is healthy and the pool is measurable . . . 13

Gate 2 — RID consumption is below the alert threshold . . . 13

Gate 3 — A dedicated OU exists and holds only project groups . . . 14

Gate 4 — Naming standard conformance . . . 14

Gate 5 — Ownership metadata present . . . 14

Gate 6 — AGDLP scope conformance . . . 14

Gate 7 — No unmanaged orphans . . . 14

Gate 8 — Token exposure within limits . . . 15

Gate 9 — Monitoring is scheduled . . . 15
Troubleshooting 15
RID Master unhealthy or unreachable . . . 15
Token-size authentication failures . . . 16
Orphaned-group detection issues . . . 16
Replication of large groups . . . 17
Discovery numbers look wrong . . . 17
Best Practices 18
Naming standards . . . 18
Dedicated OU subtree . . . 18
AGDLP and least privilege . . . 18
Lifecycle automation (the single most important control) . . . 18
Ownership metadata . . . 19
Monitoring thresholds . . . 19
Bound portfolio-spanning identities . . . 19
Keep the safety valve in reserve . . . 20
Deliverables Summary 20
What's in this package . . . 20
Outcome . . . 21
Reusable "Assessment Report" template (one-page outline) . . . 21
Appendix A — PowerShell Scripts 23

Executive Summary

The decision

Automated provisioning that creates several Active Directory security groups per project (typically a Read, a Write, and a Privileged tier) can quietly raise a scalability question — and teams almost always reach for the wrong worry. This assessment answers the question that actually matters:

> ***Is the proposed group-provisioning model sustainable over the long term*** *— without RID exhaustion, directory-scalability problems, or unmanageable operational overhead?*

The deliverable is an **architectural go/no-go with conditions of approval**, backed by measurements from your own domain rather than guesswork.

Why "will we run out of RIDs?" is the wrong first question

RIDs (Relative Identifiers) are the per-object component of every security principal's SID, drawn from a **per-domain pool of ~1.07 billion** that the RID Master issues in blocks. Because deleted objects never return their RIDs, consumption is cumulative — which is why a fast provisioning rate can *look* alarming. But the arithmetic is decisive: a per-project group model adds only a few thousand RIDs per year. Against a billion-plus pool — with a supported **extended pool** that doubles capacity in reserve — the runway is measured in **thousands of years**, even under aggressive whole-organization assumptions. **RID exhaustion is rarely the real risk.**

The risks that actually matter

At scale, a proliferating per-project group model creates exposure elsewhere, and this assessment measures each:

- **Kerberos token bloat** — every group a user belongs to rides in their logon token, which has a hard ceiling (~1,015 group SIDs) beyond which authentication fails. The exposure is **portfolio-spanning users and service accounts**, not average members.
- **Group sprawl and lifecycle debt** — with no deprovisioning, closed projects leave orphaned groups that inflate backups, replication, and audit scope, and erode least-privilege. Lifecycle automation is the single most important control.
- **Replication, backup, and administrative overhead** — modest but non-zero, and compounding if cleanup is absent.

It also reviews the **authorization architecture** (AGDLP, RBAC, nesting) so you either confirm the model's shape or right-size it against Microsoft best practice.

Business value

Instead of paying for a bespoke consulting engagement, a competent administrator or architect runs this **self-guided, tested methodology** against their own directory and produces a defensible decision: **RID lifetime projected, the real risks assessed, and a scalable design confirmed** — with a reusable report and read-only PowerShell scripts kept for every future recertification. Approve or redesign with evidence,

before the model is switched on.

Solution Overview

What this assessment is

This is a repeatable Active Directory **capacity and architecture assessment** for any provisioning model that creates **security groups per project** (or per any recurring object — per site, per customer, per team). A competent administrator or architect runs it against **their own domain** to answer one question with evidence rather than guesswork:

> ***Is the proposed group-provisioning model sustainable over the long term** — without introducing RID exhaustion, directory-scalability problems, or unmanageable operational overhead?*

The output is an **architectural go/no-go decision with conditions of approval** — not a line-of-code review. "Can we create three groups per project?" is the implementation detail; the decision this assessment supports is whether the *pattern* scales for the life of the domain.

The methodology is vendor-neutral and de-specified: every number in this package is a clearly labeled **example**. You measure your own domain's real values with the included scripts and plug them into the same formulas.

Why teams ask the wrong question first

When automated provisioning starts creating several groups per project, the reflexive worry is **"will we run out of RIDs?"** — often prompted by a platform warning about the *rate* of security-principal creation. That worry is almost always misplaced. At real-world provisioning rates the RID pool lasts far longer than the domain itself. The risks that actually bite at scale are elsewhere: **Kerberos token bloat**, **group sprawl and lifecycle debt**, and **administrative overhead**. This assessment measures RID lifetime to put that fear to rest, then spends most of its effort on the risks that genuinely matter.

RID fundamentals (the substance you are measuring)

- **Every security principal** — user, computer, or group — is assigned a unique **SID**. The

SID is the **domain SID + a RID** (Relative Identifier). RIDs come from a **per-domain pool**. (Microsoft: *Security identifiers* / SID structure.)

- The pool is owned by the **RID Master** FSMO role holder — one domain controller per domain.

The RID Master hands each DC a **block of 500 RIDs** at a time; a DC requests a fresh block when it has consumed roughly half of its current one. (Microsoft: *Managing RID issuance*.)

- The default pool is **30-bit**, giving a maximum of **2^30 − 1 = 1,073,741,823** RIDs per

domain. This is a **per-domain** ceiling, not per-forest.

- **RIDs are never reclaimed.** Deleting a group, user, or computer does **not** return its RID

to the pool. Consumption is therefore **cumulative over the entire life of the domain** — which is exactly why *rate* is the thing to reason about, not the count of live objects at any moment.

- **Safety valve — the extended RID pool.** On Windows Server 2012 and later, the domain's RID

ceiling can be raised from 2^30 to **2^31 (2,147,483,648)** by unlocking the extended pool. This is a supported, one-way operation held in reserve; it effectively doubles capacity. You should almost never need it, but its existence means even a pessimistic projection has headroom. (Microsoft: *RID pool depletion / extended RID pool*.)

What actually drives consumption

Because RIDs are cumulative and never reclaimed, consumption is driven by the **creation rate of all security principals over time**, from **every** source in the domain — not just the provisioning app under review:

- users, computers, and service accounts created by all processes;
- every group created by any tool, including this provisioning model;
- historical creation across the full life of the domain (already reflected in the current

consumed count you will measure).

The provisioning model under review adds an **incremental** burn on top of that blended organizational rate. The assessment quantifies both the app-only contribution and the blended whole-domain view.

The decision this assessment supports

At the end you produce a **go / no-go with conditions**:

- **RID lifetime** projected from *your* measured consumption and *your* provisioning rate.
- **The real risks assessed** — token bloat, sprawl/lifecycle, replication/GC, backup, admin.
- **Authorization architecture reviewed** — is the group model the right shape (AGDLP), and can

nesting/RBAC bound object growth?

- **Conditions of approval** — the guardrails (lifecycle automation, dedicated OU + naming

standard, ownership metadata, monitoring, recertification) that make an approval sustainable.

If the guardrails are in place, the typical outcome is **approve** — RID capacity is rarely the limiting factor. If they are absent, the assessment tells you exactly what to build before the model is switched on.

Requirements & Inputs

Confirm every input below before running the assessment. Each entry follows the same shape: **Description / Why / How to verify / How to implement if missing.** These are in addition to the category-level prerequisites in `global-requirements-infrastructure.md`.

R1. Active Directory read access (domain-wide)

Description: An account that can read directory objects across the domain — users, groups, computers, the RID Manager object, and organizational units. Read-only is sufficient for the discovery, validation, report, and token-exposure scripts.

Why: The assessment is a read-only diagnostic. It queries the RID pool, enumerates project groups, and inspects group memberships and token contributors. None of that requires write access (the only script that *can* change objects — the lifecycle cleanup — defaults to preview and is run separately with elevated rights).

How to verify: From a workstation with RSAT installed, run `Get-ADDomain` and `Get-ADGroup -Filter * -ResultSetSize 5`. Both must return data without an access error.

How to implement if missing: Use any standard domain user for reads; most attributes needed (group counts, `whenCreated`, `tokenGroups`, `rIDAvailablePool`) are readable by *Authenticated Users*. If your directory is hardened, request a read-only account or membership in a directory-readers role.

R2. RSAT ActiveDirectory PowerShell module

Description: The Remote Server Administration Tools **ActiveDirectory** module, providing `Get-ADDomain`, `Get-ADGroup`, `Get-ADUser`, `Get-ADObject`, etc.

Why: Every script in this package uses these cmdlets. Without the module the scripts cannot run.

How to verify: `Get-Module -ListAvailable ActiveDirectory` returns the module.

How to implement if missing: On Windows 10/11: `Add-WindowsCapability -Online -Name "Rsat.ActiveDirectory.DS-LDS.Tools~~~~0.0.1.0"`. On Windows Server: `Install-WindowsFeature RSAT-AD-PowerShell`.

R3. RID Master reachability and health

Description: The domain controller holding the **RID Master** FSMO role must be online and healthy, and reachable from where you run the discovery script.

Why: The RID pool ceiling and next-issuable RID are read from the **RID Manager$** object, and RID Master health is a gating finding — a sick RID Master blocks *all* new principal creation domain-wide, which is a more urgent problem than pool capacity.

How to verify: `netdom query fsmo` (or `Get-ADDomain | Select RIDMaster`) identifies the holder; `dcdiag /test:RidManager /v` and `repadmin /showrepl the-ridmaster-dc` confirm health.

How to implement if missing: If the current holder is unhealthy, transfer or (if it is permanently offline) seize the RID Master role to a healthy DC before proceeding. Resolve any replication errors on that DC first.

R4. The provisioning rate and groups-per-object count

Description: Two numbers that define the workload under review: **groups created per project** (or per object) and **projects created per week** (or per whatever period). Also gather the **total blended organizational RID burn per year** if you want the whole-domain projection.

Why: These drive the entire lifetime projection. Annual app burn = `groups-per-project x projects-per-week x 52`. Without them you can measure the pool but cannot project its lifetime.

How to verify: Read the provisioning tool's design (how many groups it creates per project) and its expected throughput (projects/week) from the project's own runbook or product owner. For the blended figure, sample historical creation: compare `rIDAvailablePool` consumed value across two dates (e.g. 12 months apart) and divide by the elapsed time.

How to implement if missing: If throughput is unknown, use a **worst-case** upper bound (peak projects/week) for the projection and label it as such. If historical burn is unknown, run the discovery script now, record the consumed count and date, and re-run in a known interval to derive the real annual rate.

R5. Dedicated OU and naming standard for provisioned groups

Description: A single OU subtree that contains **only** the provisioned project groups, and a deterministic naming convention that embeds the project identifier. This package uses these names verbatim throughout — adapt the domain component to your own:

- **OU:** `OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn`
- **Group name pattern:** `PRJ-projectId-ROLE (ROLE = READ, WRITE, or PRIV)` — e.g. `PRJ-1042-READ`.

Why: A dedicated OU makes every project group targetable for counting, reporting, and cleanup; the naming standard makes the project ID machine-parseable so orphan detection and growth reporting are deterministic. Without both, the report and validation scripts cannot reliably distinguish project groups from the rest of the directory.

How to verify: `Get-ADOrganizationalUnit -Identity "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn"` succeeds, and spot-checked group names match the PRJ-projectId-ROLE pattern.

How to implement if missing: Create the OU (`New-ADOrganizationalUnit -Name ProjectGroups -Path "OU=Provisioned,DC=your-domain-dn"`) and configure the provisioning tool to create groups there with the naming pattern above. If groups already exist elsewhere under inconsistent names, plan a one-time rename/move before relying on the reports.

R6. Ownership metadata on provisioned groups

Description: Each provisioned group carries owner and project-lifecycle metadata — at minimum a project ID and creation date, ideally an owner reference. Store these in `managedBy`, `info`, or a chosen extension attribute, consistently.

Why: Ownership metadata is what makes orphan detection, recertification, and cleanup **automatable** rather than manual. A group you cannot attribute to an active project is exactly the sprawl this assessment is trying to prevent.

How to verify: Sample groups with `Get-ADGroup PRJ-1042-READ -Properties managedBy,info,whenCreated` and confirm the fields are populated.

How to implement if missing: Have the provisioning tool stamp `managedBy`/`info` at creation time. Backfill existing groups from the provisioning system's records where possible.

R7. Token-size and RID monitoring tooling

Description: A monitoring platform (or scheduled script output) able to trend two signals: **RID-pool consumption** against the pool ceiling, and **Kerberos token size / group-SID count** for high-membership principals.

Why: The assessment's approval is *conditional on monitoring*. RID trend is the early-warning for the (unlikely) capacity case; token-size trend is the early-warning for the (likely) bloat case on portfolio-spanning users and service accounts.

How to verify: Confirm a scheduled job or monitoring check exists that runs the discovery and token-exposure scripts on a cadence (e.g. weekly) and alerts on thresholds.

How to implement if missing: Schedule `RIDCAP-Discovery.ps1` and `RIDCAP-TokenExposure.ps1` and route their output to your monitoring platform, alerting at **80% of the RID pool** and when a principal's group-SID count approaches the token limits (see `07-best-practices.md`).

Assessment Methodology

This is the ordered method. Run the five steps in sequence; each produces evidence that feeds the final findings. All measurement steps are **read-only**. Every number shown is a **labeled example** — substitute your own measured values.

Step 1 — Measure current RID consumption and RID Master health

Goal: establish the two facts the whole projection rests on — how much of the pool is already consumed, and whether the RID Master is healthy enough to issue RIDs at all.

1. Identify the RID Master and confirm its health (R3):

`Get-ADDomain | Select RIDMaster`, then `dcdiag /test:RidManager /v` on that DC.

1. Run `RIDCAP-Discovery.ps1`. It reads the `rIDAvailablePool` attribute from the

RID Manager$ object and decodes it. That attribute is a 64-bit value packing two 32-bit halves:

- **low 32 bits** = the **next RID to be issued** = the count already **consumed**;
- **high 32 bits** = the **pool ceiling** = the maximum RIDs (default `1,073,741,823`).

1. Record: **RIDs consumed**, **pool maximum**, **RIDs remaining**, **percent consumed**, and the

RID Master identity.

Worked example (substitute your own): a domain reports `rIDAvailablePool` decoding to **consumed = 1,478,144** against a **maximum of 1,073,741,823** → **remaining = 1,072,263,679**, **percent consumed** ≈ **0.138%**. Your numbers will differ; the script prints yours.

Step 2 — Project the pool lifetime

Goal: convert the measured consumption and your provisioning rate into a runway in years, both for the app alone and for the blended whole-domain view.

Formulas:

```
annual app burn        = groups-per-project  x  projects-per-week  x  52
years to exhaust (app) = RIDs remaining  /  annual app burn
years to exhaust (org) = RIDs remaining  /  (blended org burn/yr  +  annual app burn)
```

2a. App-only projection (worked EXAMPLE — substitute your rate)

Example workload: **3 groups per project**, worst-case **40 projects/week**. Annual app burn = `3 x 40 x 52 = 6,240 groups/year`.

Metric	Example value
Groups per project	3
Projects per week (worst case)	40
Annual app burn	**6,240 RIDs/yr**
RIDs remaining (from Step 1 example)	1,072,263,679
Years to exhaust — app alone (worst case)	≈ **171,800 years**
Years to exhaust — app alone (low case, 30/wk → 4,680/yr)	≈ 229,100 years

2b. Blended whole-domain projection (worked EXAMPLE)

RIDs are consumed by **every** principal ever created, not just this app. Add the app burn to an assumed organization-wide annual burn (measure yours per R4; these rows are illustrative):

Assumed org burn/yr	+ app burn (6,240)	Years to exhaust remaining
50,000	56,240	≈ 19,070 years
100,000	106,240	≈ 10,090 years
250,000	256,240	≈ 4,185 years
500,000	506,240	≈ 2,118 years

How to read this: even under an aggressive blended burn, the runway is measured in **millennia**, and the **extended RID pool** (2^31) doubles it if ever needed. The incremental burn from a per-project group model is statistically negligible against the remaining pool.

Typical finding: *RID exhaustion risk = Very Low.* This is not, on its own, a basis to block or approve the design — which is why Steps 3–4 exist.

Step 3 — Evaluate the non-RID risks (the ones that actually matter)

Assess each of the following against your measured workload. These, not RIDs, are where a per-project group model creates real exposure at scale.

3.1 Kerberos token bloat — usually the highest practical risk

• A user's Kerberos ticket (**PAC**) carries the **SID of every group they belong to** — including, under AGDLP, the domain-local resource-group SIDs, plus any SID history.

• Default **MaxTokenSize = 48,000 bytes** (Server 2012+), and there is a hard ceiling of roughly **1,015 group SIDs** before authentication fails outright. (Microsoft: *MaxTokenSize / token-size problems*.)

• **Where it bites:** a **user, service account, or automation identity attached to many projects.** At 3 groups/project, an identity on 100 projects carries **~300 project-group SIDs** before its baseline enterprise groups. Portfolio-spanning PMs, power users, and service accounts are the exposure — not the average member.

• **Measure it:** run `RIDCAP-TokenExposure.ps1` to count `tokenGroups` (effective SID count, including nesting) for the identities that span the most projects, and compare against thresholds.

3.2 Group sprawl and lifecycle debt — usually the highest operational risk

• At the example rate, 6,240 groups/year with **no deprovisioning** = **62,400 groups in 10 years** from this app alone.

• Without automated cleanup, completed and abandoned projects leave **orphaned groups** that inflate backups, replication, search, audit scope, and erode least-privilege over time.

• **Measure it:** run `RIDCAP-Report.ps1` to count project groups by project ID and flag those whose project is closed (not on your active-projects list).

• **The single most important guardrail** is lifecycle automation tied to project state (see `07-best-practices.md`). Its absence, not RID capacity, is what makes a model unsustainable.

3.3 Replication and Global Catalog

• Groups and their memberships replicate to all DCs; universal groups and GC-replicated attributes add inter-site traffic. At thousands of small groups per year this is **modest but non-zero**.

• **Large-membership** groups matter more than group *count*. Ensure the forest is at a functional level with **linked-value replication** (Windows Server 2003+), so a membership change replicates **per member** rather than re-sending the whole member list.

3.4 Backup / DIT size

• Every object adds to the **NTDS.DIT** and system-state backup. Thousands/year is small relative to a typical enterprise directory, but it **compounds with 3.2** if cleanup is absent.

3.5 Administration and governance overhead

• Naming collisions, ownership ambiguity, unclear scope, and audit fatigue all scale with object count. This is contained by naming standards, a dedicated OU subtree, and ownership metadata.

3.6 SYSVOL — not impacted

• These are **permission/security groups**, not GPO objects. SYSVOL/GPO scale is **not** a factor

unless the groups are used for GPO security filtering. For a resource-permission model they are not, so SYSVOL is out of scope.

Step 4 — Review the authorization architecture

Goal: confirm the group model is the right *shape*, and identify whether nesting/RBAC can bound object growth without sacrificing clarity.

4.1 Apply AGDLP; do not flatten

Microsoft's canonical pattern is **A** → **G** → **DL** → **P**: **Accounts** → **Global groups (roles)** → **Domain Local groups (resource permissions)** → **Permissions.**

• Map the **per-project Read/Write/Privileged groups to Domain Local "resource" groups** applied to the project's file shares / SharePoint / resources.

• Add users via **Global "role" groups** nested into those resource groups.

• This keeps ACLs clean, supports multi-domain/forest, and localizes churn to **membership** changes (linked-value replication) instead of ACL rewrites.

• Validate conformance with `RIDCAP-Validation.ps1` (group scope, dedicated OU, naming, ownership metadata).

4.2 Options to bound object growth (choose deliberately)

Approach	Effect	Trade-off
Keep 3 resource groups/project (default)	Cleanest least-privilege; RID-safe	Highest object count (acceptable at these rates)
1 project group + roles via nested global groups	Fewer per-project objects	Permission logic moves into nesting; less transparent ACLs
RBAC via reused enterprise role groups + resource groups	Reuses existing roles; fewer new objects	Requires a maintained role catalog
Entra ID / dynamic groups	Good for **cloud** resources	Does **not** set on-prem NTFS/SharePoint-Server ACLs — not a substitute for AD groups on file resources

Guidance: retain the multi-group-per-project resource model unless **measured token bloat** (3.1) on portfolio-spanning identities forces consolidation. Collapse groups only to solve a demonstrated token problem, accepting the loss of ACL transparency.

Step 5 — Produce findings and conditions of approval

Assemble the evidence into a decision:

1. **State the RID finding** with your measured numbers (Step 1–2): consumed %, remaining, projected

lifetime (app-only and blended). Typically *Very Low* risk.

1. **State the non-RID findings** (Step 3): token-bloat exposure for named high-membership identities, current orphan/sprawl count, replication/backup notes.

1. **State the architecture finding** (Step 4): AGDLP conformance and whether growth-bounding is warranted.

1. **Record the decision as go / no-go with conditions.** The standard outcome is **approve, conditional on the guardrails** in `07-best-practices.md`: lifecycle automation, dedicated OU + naming standard, AGDLP implementation, ownership metadata, RID/token/orphan monitoring, and access recertification.

1. **Set re-review triggers:** revisit if per-project group count increases, if any identity approaches token limits, or if the RID trend alert fires at ≥ 80% of the pool.

Capture all of this in the reusable **Assessment Report** template outlined in `08-deliverables-summary.md`.

Testing & Validation

This section validates that the assessment's **monitoring and guardrails are actually in place** — not merely recommended. Each check has an explicit **pass/fail gate**. Run these after Step 5 of the methodology and re-run them on every recertification cycle. All checks are read-only.

How to run the gates

Run the discovery, validation, report, and token-exposure scripts, then evaluate each gate below against their output:

```
powershell -ExecutionPolicy Bypass -File .\RIDCAP-Discovery.ps1
powershell -ExecutionPolicy Bypass -File .\RIDCAP-Validation.ps1     -ProjectGroupsOU "OU=ProjectGroups,OU
powershell -ExecutionPolicy Bypass -File .\RIDCAP-Report.ps1         -ProjectGroupsOU "OU=ProjectGroups,OU
powershell -ExecutionPolicy Bypass -File .\RIDCAP-TokenExposure.ps1  -Top 25
```

Gate 1 — RID Master is healthy and the pool is measurable

Check: `RIDCAP-Discovery.ps1` returns a RID Master identity, a decoded consumed/maximum/remaining, and a percent consumed.

- **PASS:** all fields populate, RID Master is reachable, and `dcdiag /test:RidManager` reports no errors on the holder.

- **FAIL:** the script cannot read `rIDAvailablePool`, or the RID Master is unreachable/unhealthy.

Resolve per `06-troubleshooting.md` before trusting any projection.

Gate 2 — RID consumption is below the alert threshold

Check: percent consumed from `RIDCAP-Discovery.ps1`.

- **PASS:** percent consumed **< 80%** of the pool ceiling.
- **FAIL:** ≥ **80%** — investigate the burn source and plan the extended RID pool. (For context, a healthy directory running a per-project group model typically reads a fraction of one percent.)

Gate 3 — A dedicated OU exists and holds only project groups

Check: `RIDCAP-Validation.ps1` confirms the OU `OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn` exists and that the groups within it are the provisioned project groups.

- **PASS:** the OU resolves and contains the project groups; no unrelated principals are mixed in.
- **FAIL:** OU missing, or project groups scattered outside it. Create/consolidate per requirement R5.

Gate 4 — Naming standard conformance

Check: `RIDCAP-Validation.ps1` reports the count of groups that do **not** match `PRJ-projectId-ROLE (ROLE = READ, WRITE, or PRIV)`.

- **PASS:** non-conforming count = **0** (100% conformance).
- **FAIL:** any non-conforming names — reporting and cleanup cannot reliably parse project IDs. Rename to standard.

Gate 5 — Ownership metadata present

Check: `RIDCAP-Validation.ps1` reports the percentage of project groups missing ownership metadata (`managedBy`/`info`) or a creation date.

- **PASS:** ≥ **95%** of project groups carry ownership metadata and a creation date.
- **FAIL:** below 95% — orphan detection and recertification cannot be automated. Backfill metadata.

Gate 6 — AGDLP scope conformance

Check: `RIDCAP-Validation.ps1` reports each project group's scope.

- **PASS:** project resource groups are **Domain Local** (per AGDLP), and members arrive via nested Global role groups.
- **FAIL:** resource groups created as Global/Universal, or users added directly to resource groups. Rework toward AGDLP per methodology Step 4.

Gate 7 — No unmanaged orphans

Check: `RIDCAP-Report.ps1` flags project groups whose project ID is not on the supplied active-projects list.

- **PASS:** orphan count is **0**, or every flagged group is already in the lifecycle disable/stage/delete pipeline.

- **FAIL:** orphans exist outside the lifecycle pipeline — sprawl is accumulating. Run the lifecycle cleanup (preview first).

Gate 8 — Token exposure within limits

Check: `RIDCAP-TokenExposure.ps1` lists the highest-membership principals and their effective group-SID counts.

- **PASS:** no principal's group-SID count exceeds the **warning threshold** (default 900, below the ~1,015 hard ceiling), and none exceeds MaxTokenSize modeling.
- **FAIL:** any principal at/over the warning threshold — scope that identity's project membership down or consolidate groups per methodology Step 4.

Gate 9 — Monitoring is scheduled

Check: confirm `RIDCAP-Discovery.ps1` and `RIDCAP-TokenExposure.ps1` run on a schedule and feed your monitoring platform with alert thresholds configured (R7).

- **PASS:** both are scheduled and alerting (RID at 80%; token at the warning threshold).
- **FAIL:** either is run only ad hoc. Schedule them — the approval is conditional on monitoring.

Overall result: the assessment's guardrails are considered validated only when **Gates 1–9 all PASS**. Any FAIL is a condition of approval that has not yet been met; record it and remediate before the provisioning model is treated as production-approved.

Troubleshooting

Common problems encountered while running the assessment, and how to resolve them. Symptoms are grouped by the risk area they belong to.

RID Master unhealthy or unreachable

Symptoms: `RIDCAP-Discovery.ps1` cannot read `rIDAvailablePool`; new principal creation fails domain-wide with *"the directory service was unable to allocate a relative identifier"*; DCs cannot obtain new RID blocks.

Why it matters: the RID Master is the single issuer of RID blocks. If it is down or its own pool is exhausted/invalidated, **no** DC can create users, computers, or groups — a more urgent outage than any capacity projection.

Diagnose:

- Identify the holder: `Get-ADDomain | Select RIDMaster` or `netdom query fsmo`.
- Health: `dcdiag /test:RidManager /v` on the holder; check the **Directory Service** event log for RID-allocation events; `repadmin /showrepl the-ridmaster-dc` for replication errors.

Resolve:

- If the holder is merely unhealthy, fix its replication first, then re-test.
- If it is permanently offline, **transfer** the role to a healthy DC

(`Move-ADDirectoryServerOperationMasterRole -Identity the-healthy-dc -OperationMasterRole RIDMaster`), or **seize** it only if the old holder will never return.

- If the pool ceiling is genuinely near the 30-bit maximum (an 80%+ Discovery reading), plan the

extended RID pool (raises the ceiling to 2^31) as the supported, one-way safety valve — after confirming the burn source.

Token-size authentication failures

Symptoms: specific users or service accounts fail Kerberos authentication while others succeed; errors such as *KDC_ERR_MORE_DATA*, HTTP 400 "bad request / header too large" on Kerberos-backed web apps, or logon failures for portfolio-spanning accounts. `RIDCAP-TokenExposure.ps1` flags the account WARN/CRITICAL.

Why it matters: every group SID rides in the Kerberos PAC. Past MaxTokenSize (default 48,000 bytes) and the ~1,015 group-SID hard ceiling, authentication fails outright. This is the token-bloat risk the assessment predicts for identities attached to many projects.

Diagnose:

- Run `RIDCAP-TokenExposure.ps1 -Identity the-account` to get the effective group-SID count.
- Compare against the WARN (900) / CRITICAL (~1,015) thresholds.

Resolve:

- **Scope the identity down** — remove it from projects it no longer needs (service accounts are the

worst offenders; give each automation only the projects it truly touches).

- **Consolidate groups** for that use case per methodology Step 4 (role reuse / nesting) — but only to

fix a *measured* problem.

- **Avoid deep nesting** that inflates the effective SID set.
- As a stopgap only, MaxTokenSize can be raised, but this treats the symptom, not the sprawl — reduce

membership as the real fix.

Orphaned-group detection issues

Symptoms: `RIDCAP-Report.ps1` reports zero orphans when you expect many, flags active groups as orphans, or cannot parse project ids.

Why it matters: orphan detection is how you find lifecycle debt. If it is unreliable, sprawl accumulates invisibly.

Diagnose & resolve:

- **Names don't match the standard.** If groups aren't named `PRJ-projectId-ROLE (ROLE = READ, WRITE, or PRIV)`, the

project id can't be parsed. Confirm with `RIDCAP-Validation.ps1` (naming check) and rename to standard. Adjust `-ProjectIdField` only if your id sits in a different hyphen-delimited position.

- **Active-projects list is wrong.** Ensure `active-projects.txt` has **one project id per line** and

the ids match exactly what's embedded in the group names (comparison is case-insensitive but the id strings must correspond). A stale list makes live projects look orphaned.

- **Groups live outside the OU.** `RIDCAP-Report.ps1` only searches the supplied OU. Consolidate all

project groups into `OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn` (requirement R5).

Replication of large groups

Symptoms: membership changes on high-count groups cause noticeable inter-site replication traffic or lag; GC replication spikes.

Why it matters: at scale, **large-membership** groups matter more than the raw count of small groups. Without linked-value replication, a single membership change re-sends the entire member list.

Diagnose & resolve:

- Confirm the forest functional level supports **linked-value replication** (Windows Server 2003+ so

membership replicates **per member**, not whole-list). Raise the functional level if it is lower.

- Prefer many small resource groups over a few very large ones where the design allows; the per-project

model already favors this.

- Monitor replication with `repadmin /showrepl` and `repadmin /replsummary`; investigate any DC with

persistent backlog.

- Keep universal-group membership churn modest, since universal group membership is GC-replicated.

Discovery numbers look wrong

Symptoms: consumed count seems implausibly high or low, or percent-consumed is unexpected.

Resolve:

- Remember consumption is **cumulative and never reclaimed** — a high count on an old domain with

historical bulk creation is normal and not a leak.

- Re-run `RIDCAP-Discovery.ps1` against the **RID Master** specifically to rule out a stale replica.

- Verify the decode: low 32 bits = consumed/next RID, high 32 bits = pool ceiling

(default 1,073,741,823). A ceiling of ~2.1 billion means the **extended RID pool** was already unlocked.

Best Practices

Field-proven practices for running a per-project group-provisioning model sustainably. These are the **conditions of approval** in operational form — implement them and the model stays healthy for the life of the domain.

Naming standards

- Use a **deterministic, parseable** group name that embeds the project ID and the role:

`PRJ-projectId-ROLE (ROLE = READ, WRITE, or PRIV)` (e.g. `PRJ-1042-READ`, `PRJ-1042-WRITE`, `PRJ-1042-PRIV`). Reuse this exact pattern everywhere — reports and cleanup parse the project ID from the second hyphen-delimited field.

- Keep the prefix (`PRJ-`) reserved for provisioned project groups only, so a prefix match cleanly

separates them from the rest of the directory.

- Keep names within length limits and collision-free by construction (the project ID guarantees

uniqueness). Never encode mutable data (owner name, department) into the name — put that in metadata.

Dedicated OU subtree

- Place every provisioned group in one subtree:
  `OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn`.

A dedicated OU makes counting, reporting, delegation, and cleanup a single-scope operation.

- Delegate write permissions on that OU to the provisioning service account only — nothing else

should create objects there.

- Never mix users, computers, or GPO-linked groups into this OU.

AGDLP and least privilege

- Implement **A** → **G** → **DL** → **P**: project **resource** groups are **Domain Local**; users join via

Global role groups nested into the resource groups. Apply the Domain Local groups directly to the resource ACLs.

- Keep the three privilege tiers distinct (Read / Write / Privileged) — do not collapse them just to

reduce object count; collapse only to solve a **measured** token-bloat problem.

- Recertify **Privileged** membership on a fixed cadence (e.g. quarterly); privileged access drifts

faster than any other tier.

Lifecycle automation (the single most important control)

- Tie group lifetime to **project state**. On project close/archive, run a timed pipeline:

disable → **stage** → **delete** (e.g. **30 / 60 / 90 days**). This converts an ever-growing pile into a steady state and is what makes the whole model sustainable.

1. **Disable** (day 0 of closure): mark the group, remove it from active use, keep it recoverable.
2. **Stage** (e.g. day 30): move to a quarantine OU / clear membership, retain for audit.
3. **Delete** (e.g. day 90): remove the object after the retention window.

• Preview every cleanup run with `RIDCAP-LifecycleCleanup.ps1` (defaults to `-WhatIf`) before executing, and always drive it from the authoritative **active-projects list**.

Ownership metadata

• Stamp each group at creation with **owner**, **project ID**, and **creation date** in `managedBy` / `info` (or a chosen extension attribute), consistently.

• This is what makes orphan detection, recertification, and cleanup automatable rather than manual. A group you cannot attribute to an active project is sprawl by definition.

Monitoring thresholds

Schedule the read-only scripts and alert on these thresholds:

Signal	Source script	Warn	Critical
RID pool consumed	`RIDCAP-Discovery.ps1`	**80%** of pool ceiling	90% (plan extended RID pool)
Group-SID count per principal	`RIDCAP-TokenExposure.ps1`	**900 SIDs**	~1,015 (hard auth-failure ceiling)
Orphaned project groups	`RIDCAP-Report.ps1`	any orphan outside lifecycle pipeline	orphan count trending up week-over-week
Projected RID lifetime	`RIDCAP-Report.ps1`	< 50 years	< 10 years

• Alerting on RID at **80%** gives enormous lead time — at typical rates the pool sits at a fraction of one percent, so an 80% reading means something abnormal is burning RIDs.

• Watch **token size for portfolio-spanning identities** (PMs, power users, service accounts). They, not average members, are the token-bloat exposure.

Bound portfolio-spanning identities

• Scope **service and automation accounts** to only the projects they truly need. An automation identity attached to the whole portfolio is the fastest path to token-size auth failures.

• Prefer **role-based** access (a maintained catalog of enterprise role groups) over adding an identity to every project's groups individually.

Keep the safety valve in reserve

• Know that the **extended RID pool** (2^31) exists as a one-way, supported doubling of capacity on Server 2012+. Treat it as a genuine last resort triggered by the 80% alert — not a routine step, and never something to unlock preemptively.

Deliverables Summary

What's in this package

A complete, reusable Active Directory capacity-and-architecture assessment you run against your own domain — documentation plus five read-only-by-default PowerShell scripts.

Documentation

• **Solution Overview** — RID / RID Master / per-domain pool fundamentals, what drives consumption, and the go/no-go decision this assessment supports.

• **Requirements & Inputs** — every prerequisite with Description / Why / Verify / Implement-if-missing.

• **Assessment Methodology** — the five-step method: measure RID consumption, project pool lifetime (app-only + blended, with worked example tables), evaluate non-RID risks, review authorization architecture, and produce findings + conditions of approval.

• **Testing & Validation** — nine pass/fail gates confirming monitoring and guardrails are in place.

• **Troubleshooting** — RID Master unhealthy, token-size auth failures, orphan detection, large-group replication.

• **Best Practices** — naming, dedicated OU, AGDLP, lifecycle automation, monitoring thresholds.

• **Deliverables Summary** — this document, plus the reusable Assessment Report template outline.

Scripts (`scripts/`)

Script	Mode	Purpose
`RIDCAP-Discovery.ps1`	read-only	RID-pool consumption, % consumed, remaining, RID Master identity/health
`RIDCAP-Validation.ps1`	read-only	Guardrail validation: dedicated OU, naming conformance, ownership metadata, scope
`RIDCAP-Report.ps1`	read-only	Project-group growth & orphan report by project ID + projected lifetime
`RIDCAP-TokenExposure.ps1`	read-only	Kerberos token-size / group-SID-count exposure for high-membership principals

Script	Mode	Purpose
`RIDCAP-LifecycleCleanup.ps1`	preview by default (`-WhatIf`)	Disable→stage→delete of orphaned project groups per an active-projects list

Outcome

An evidence-based **go/no-go on the group-provisioning model**: RID lifetime projected from your own measurements, the real risks (token bloat, sprawl, lifecycle) assessed, a scalable AGDLP/RBAC design confirmed, and a reusable report and scripts you keep and re-run on every recertification cycle.

Reusable "Assessment Report" template (one-page outline)

Fill this in from the methodology outputs. Every value comes from your own measured data.

1. Header

• Domain / scope · Assessment type: AD capacity + authorization-architecture review · Date · Reviewer.

2. Restated requirement

• One sentence: *is the proposed per-object group-provisioning model sustainable long-term without RID exhaustion, scalability issues, or unmanageable operational risk?*

3. Current state & workload

• Groups per project/object · Projects/objects per period · Derived groups/year · Any prior/retired model.

4. RID measurement (Step 1)

• RIDs consumed · Pool maximum · RIDs remaining · Percent consumed · RID Master identity & health.

5. RID projection (Step 2)

• Annual app burn · Years-to-exhaust (app-only, worst & low case) · Blended whole-domain table · Extended-pool note · **Finding** (e.g. RID risk = Very Low).

6. Non-RID risk findings (Step 3)

• Token bloat (named high-membership identities + SID counts) · Sprawl/lifecycle (current group count, orphan count) · Replication/GC · Backup/DIT · Admin overhead · SYSVOL (not impacted).

7. Authorization-architecture findings (Step 4)

• AGDLP conformance · Growth-bounding options considered · Recommendation on model shape.

8. Decision & conditions of approval (Step 5)

• Go / No-go · The guardrail conditions (lifecycle automation, dedicated OU + naming, AGDLP, ownership metadata, RID/token/orphan monitoring, recertification) · Re-review triggers.

9. Sign-off

• Reviewed by · Date · Next review (on material design change or annually).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

RIDCAP-Discovery.ps1

Purpose: Reads the domain RID pool from the RID Manager object and decodes it into RIDs consumed, pool maximum, RIDs remaining, and percent consumed. Also identifies the RID Master FSMO holder so its health can be confirmed. Read-only. Run this against your-domain to establish the numbers every projection depends on. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RIDCAP-Discovery.ps1 Requires the RSAT ActiveDirectory module and domain read access. Optional: -AlertThresholdPercent 80 to change the warning gate. EXPECTED RESULTS A summary of RIDMaster, RIDsConsumed, RIDMaximum, RIDsRemaining, PercentConsumed. PASS when PercentConsumed is below the alert threshold (default 80%); FAIL at/above it. .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

How to run: powershell -ExecutionPolicy Bypass -File .\RIDCAP-Discovery.ps1 Requires the RSAT ActiveDirectory module and domain read access. Optional: -AlertThresholdPercent 80 to change the warning gate. EXPECTED RESULTS A summary of RIDMaster, RIDsConsumed, RIDMaximum, RIDsRemaining, PercentConsumed. PASS when PercentConsumed is below the alert threshold (default 80%); FAIL at/above it. .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

RIDCAP-LifecycleCleanup.ps1

Purpose: Drives the lifecycle pipeline for project groups whose project is no longer active: disable, then stage (move to a quarantine OU), then delete after a retention window. Defaults to preview (-WhatIf) so nothing changes until you explicitly commit. Run it against your-domain with an active-projects list to reclaim sprawl safely. HOW TO RUN Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\RIDCAP-LifecycleCleanup.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" -ActiveProjectsFile .\active-projects.txt -Stage Disable Commit a stage: add -Commit (removes -WhatIf). Stages: Disable, Stage, Delete. Requires the RSAT ActiveDirectory module and write rights on the OU for -Commit. EXPECTED RESULTS A list of orphaned groups and the action that WOULD run per stage. With -Commit the chosen stage is applied. Deletes only touch groups already in the quarantine OU. .NOTES White-label — the repeatable way. Preview-only by default; commit is explicit and staged. #>

How to run: Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\RIDCAP-LifecycleCleanup.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" -ActiveProjectsFile .\active-projects.txt -Stage Disable Commit a stage: add -Commit (removes -WhatIf). Stages: Disable, Stage, Delete. Requires the RSAT ActiveDirectory module and write rights on the OU for -Commit. EXPECTED RESULTS A list of orphaned groups and the action that WOULD run per stage. With -Commit the chosen stage is applied. Deletes only touch groups already in the quarantine OU. .NOTES White-label — the repeatable way. Preview-only by default; commit is explicit and staged. #>

RIDCAP-Report.ps1

Purpose: Counts provisioned project groups by parsed project id, flags groups whose project is not on the supplied active-projects list (orphans), and computes a projected RID pool lifetime from a supplied annual

burn rate. Read-only. Run it against your-domain to quantify sprawl and to see the runway in years. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RIDCAP-Report.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" -ActiveProjectsFile .\active-projects.txt -AnnualBurn 6240 active-projects.txt = one active project id per line. Requires the RSAT ActiveDirectory module and domain read access. EXPECTED RESULTS Total project-group count, top project ids by group count, an orphan list, and a projected years-to-exhaust from remaining RIDs divided by the supplied annual burn. .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

How to run: powershell -ExecutionPolicy Bypass -File .\RIDCAP-Report.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" -ActiveProjectsFile .\active-projects.txt -AnnualBurn 6240 active-projects.txt = one active project id per line. Requires the RSAT ActiveDirectory module and domain read access. EXPECTED RESULTS Total project-group count, top project ids by group count, an orphan list, and a projected years-to-exhaust from remaining RIDs divided by the supplied annual burn. .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

RIDCAP-TokenExposure.ps1

Purpose: Enumerates the effective group-SID count (tokenGroups, including nested membership) for high-membership users and service accounts, and flags any approaching the token limits. Group SIDs ride in the Kerberos PAC; too many cause authentication to fail. Read-only. Run it against your-domain to find portfolio-spanning identities before they break. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RIDCAP-TokenExposure.ps1 -Top 25 Optionally scope to a set of names: -Identity user1,svc-app2 Requires the RSAT ActiveDirectory module and domain read access. EXPECTED RESULTS A table of principals ranked by group-SID count with WARN/CRITICAL flags. PASS when no principal reaches the warning threshold (default 900; hard ceiling is about 1,015). .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

How to run: powershell -ExecutionPolicy Bypass -File .\RIDCAP-TokenExposure.ps1 -Top 25 Optionally scope to a set of names: -Identity user1,svc-app2 Requires the RSAT ActiveDirectory module and domain read access. EXPECTED RESULTS A table of principals ranked by group-SID count with WARN/CRITICAL flags. PASS when no principal reaches the warning threshold (default 900; hard ceiling is about 1,015). .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

RIDCAP-Validation.ps1

Purpose: Checks that the approval guardrails are in place: a dedicated OU holding the project groups, conformance to the PRJ-project-id-role naming standard, correct AGDLP group scope (Domain Local resource groups), and presence of ownership metadata. Read-only. Point it at your-domain project-groups OU and review the pass/fail summary. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\RIDCAP-Validation.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" Requires the RSAT ActiveDirectory module and domain read access. Optional: -MetadataPassPercent 95 to change the ownership-metadata gate. EXPECTED RESULTS Per-check PASS/FAIL for OU existence, naming conformance, group scope, and metadata coverage. Overall PASS only when every check passes. .NOTES White-label — the repeatable way. Read-only against Active Directory. #>

How to run: powershell -ExecutionPolicy Bypass -File .\RIDCAP-Validation.ps1 -ProjectGroupsOU "OU=ProjectGroups,OU=Provisioned,DC=your-domain-dn" Requires the RSAT ActiveDirectory module and domain read access. Optional: -MetadataPassPercent 95 to change the ownership-metadata gate. EXPECTED RESULTS Per-check PASS/FAIL for OU existence, naming conformance, group scope, and metadata coverage. Overall PASS only when every check passes. .NOTES White-label — the repeatable

way. Read-only against Active Directory. #>